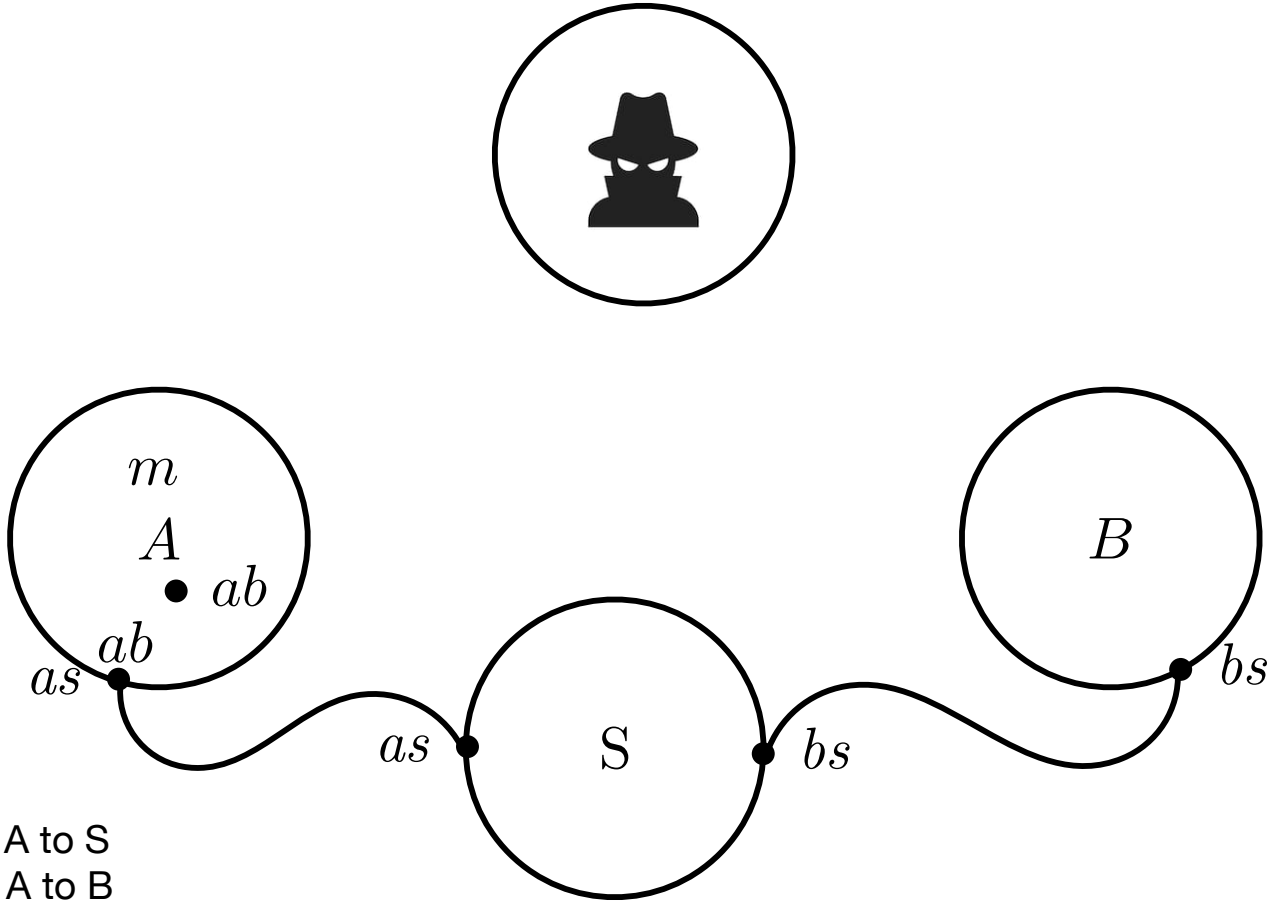
# 0-CFA for process algebras

# Name mobility

- You should have seen CCS
- In CCS communication is statically defined: channels cannot be transmitted / received
- What if channels are treated as ordinary values?
- Can we extend CCS to communicate communication means?
- Yes, this leads us to $\pi$-calculus

# π-calculus

# Name mobility

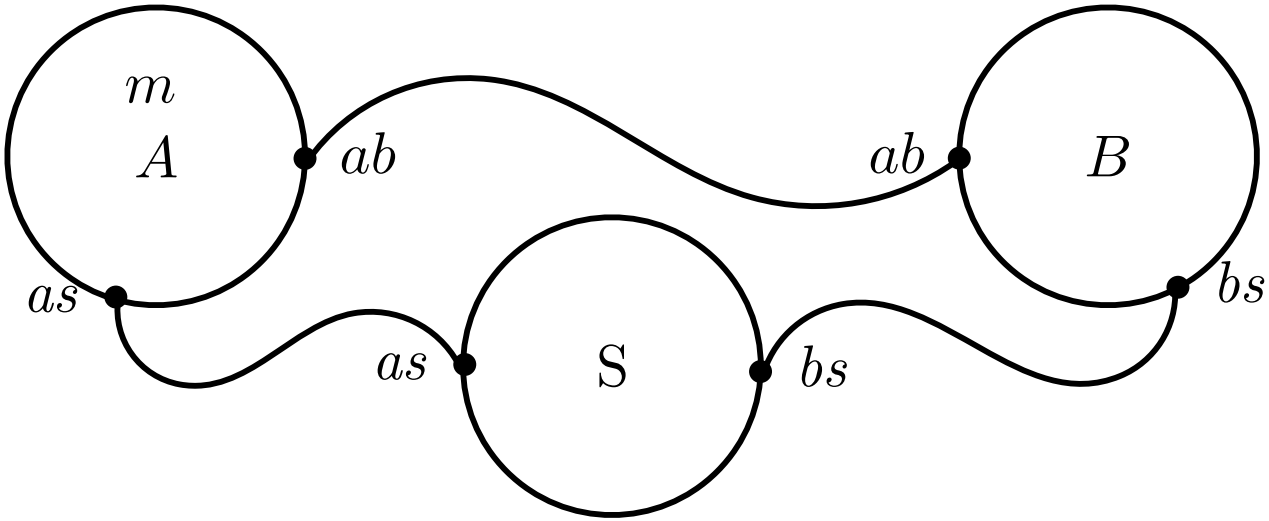



**Legenda**:
m: message
as: secure channel from A to S
bs: secure channel from A to B
ab: channel from A to B

# Name mobility

$(\nu\ as)(\nu\ bs)(\boxed{(\nu\ ab)\ A} \mid B \mid S)$

$(\nu\ as)(\nu\ bs)(\boxed{(\nu\ ab)(A \mid B)} \mid S)$



# π-calculus: syntax

Prefixes

$\pi ::= \overline{u}\langle v \rangle$ // send $v$ on $u$

$\mid\ u(x)$ // receive on $u$ and store in $v$

$\mid\ \tau$ // internal action

Processes

often omitted

$P ::= \mathbf{0}$ // inactive process

$\mid\ \pi\,.\,P$ // action prefix

$\mid\ [x = y]P$ // matching (like, if $x = y$ then $P$)

$\mid\ (\nu n)P$ // restriction (like $n$ only known in $P$)

sometimes written $P \setminus n$

$\mid\ \Sigma_{i \in I}\ \pi_i\,.\,P_i$ // non deterministic, guarded choice

nullary sum: $\mathbf{0}$

binary sum: $\pi_1\,.\,P_1 + \pi_2\,.\,P_2$

$\mid\ P_1 \mid P_2$ // parallel composition

$\mid\ !P$ // replication (like $P \mid P \mid P \mid P \mid \ldots$)

# π-calculus: the idea

- Communication primitives: simple and powerful
- Scoping Rules: explicitly control the access to channels and to data

send $z$ on $x$

receive $y$ from $x$

**Communication (on channel x)**

$$\bar{x}\langle z\rangle . P \mid x(y) . Q \rightarrow P \mid Q[z/y]$$

$y$ has been substituted for $z$ in the second process

**Extrusion of z**

$$(\nu z)\ (\bar{x}\langle z\rangle . P) \mid x(y) . Q \rightarrow (\nu z)\ (P \mid Q[z/y])$$

- $y$ has been substituted for $z$ in the second process
- the scope of $z$ has been extended to include both processes

# Free and bound names

A central component of the $\pi$-calculus is that of names
The only binders are input prefix and name restriction

$$\pi ::= \overline{u}\langle v \rangle \mid u(v) \mid \tau \mid \dots$$

free names

bound variable

$$P ::= \dots \mid (\nu\ n) \mid \dots$$

bound constant

disjoint union not present in the original $\pi$-calculus

$$\text{Name} = \text{Const} \uplus \text{Var}$$

The notion of free and bound constants and variables are defined as expected:
$fc(P)$ and $fv(P)$

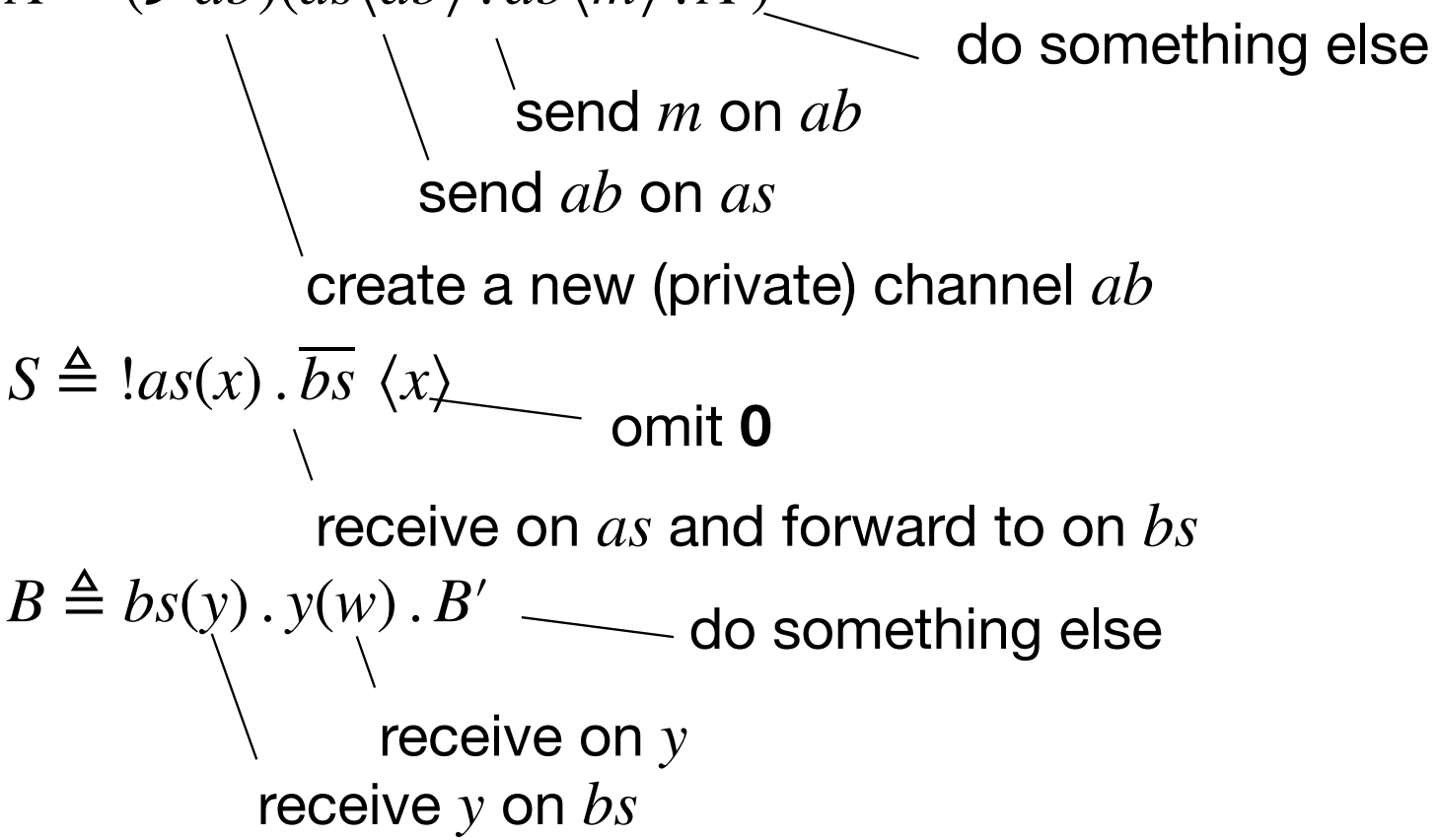
# π-calculus: syntax details

- The notion of free and bound names (constants and variables) are defined as usual: we focus on $fc(P)$ and $fv(P)$
- The notion of capture avoiding substitution is defined as usual: $P[y/x]$
- Processes are taken up-to alpha-renaming of bound names:
  - $u(y).P = u(z).(P[z/y])$ if $z \notin fv(P)$
  - $(\nu n).P = (\nu n').(P[n'/n])$ if $n' \notin fc(P)$
- We focus on closed processes, i.e., process $P$ such that $fc(P) = fv(P) = \emptyset$

# Examples

$$(\nu\ as)(\nu\ bs)(A \mid S \mid B)$$

$A \triangleq (\nu\ ab)(\overline{as}\langle ab\rangle\ .\ \overline{ab}\langle m\rangle\ .\ A')$

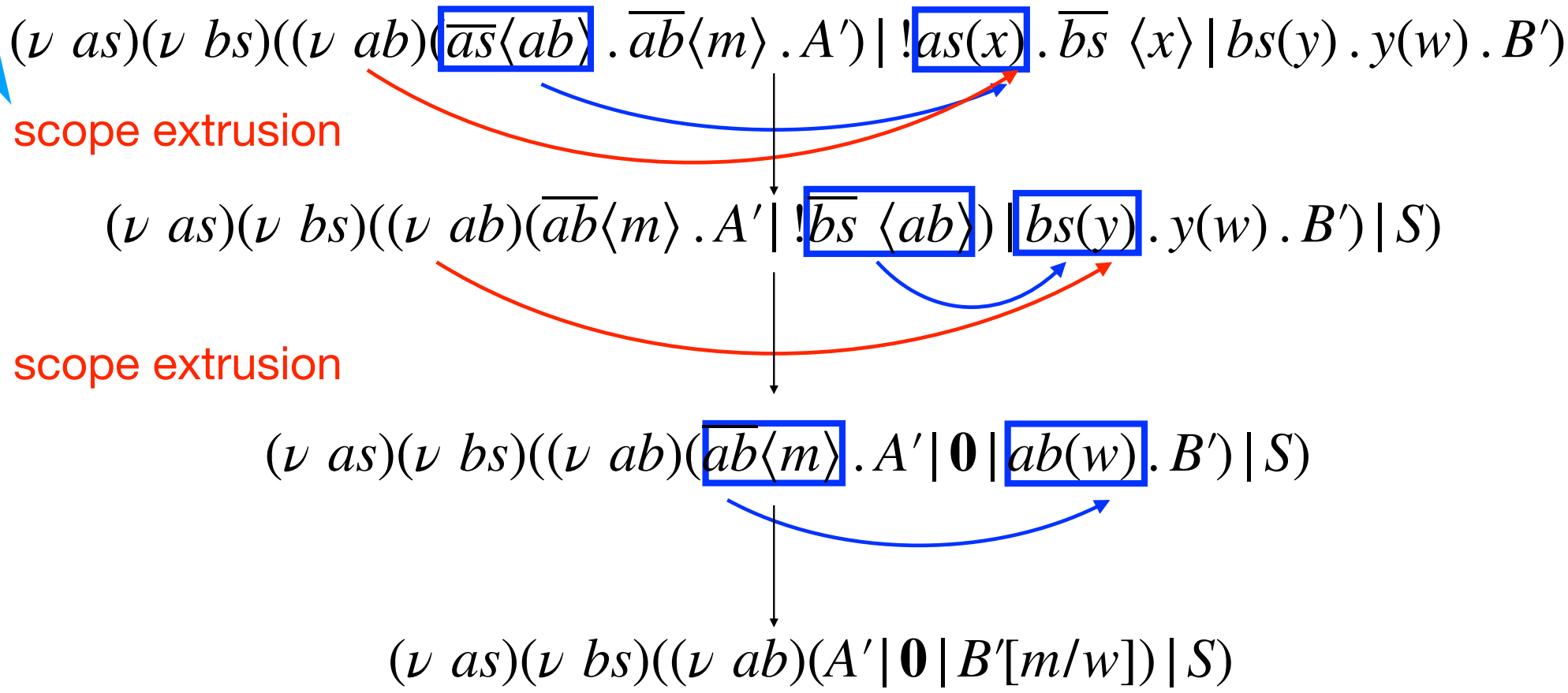



$S \triangleq\ !as(x)\ .\ \overline{bs}\ \langle x\rangle$

$B \triangleq bs(y)\ .\ y(w)\ .\ B'$

# Example

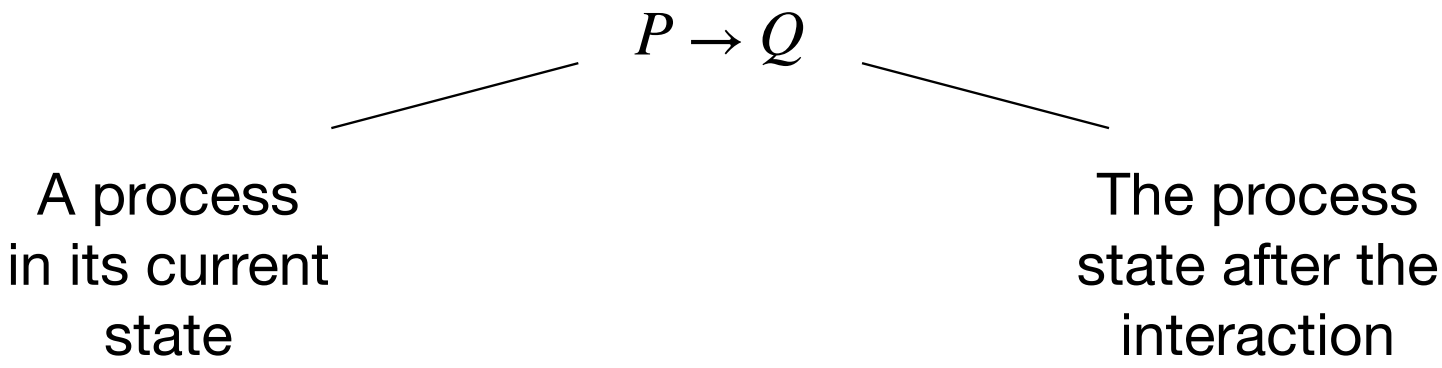
the scope of $ab$ is extended to include both processes



# π-calculus: reduction semantics

We define the semantics by:

- a structural congruence relation ≡, and
- a reduction relation →



# Structural congruence

Abelian monoid laws for parallel

$(P \mid Q) \mid R \equiv P \mid (Q \mid R)$

$P \mid Q \equiv Q \mid P$

$P \mid \mathbf{0} \equiv \mathbf{0}$

Scope laws

$(\nu n)(\nu m)P \equiv (\nu m)(\nu n)P$

$(\nu n)\mathbf{0} \equiv \mathbf{0}$

$(\nu n)(P \mid Q) \equiv (\nu m)P \mid Q$ if $n \notin fc(Q)$

Matching

$[x = x]P \equiv P$

Summands can be permuted in $\Sigma_{i \in I} \pi_i . P_i$

Unfolding of replication

$!P \equiv P \mid !P$

$\alpha$-renaming

$P \equiv_\alpha Q \Rightarrow P \equiv Q$

# Reduction semantics

$$[Com]\ (\overline{n}\langle m\rangle . P + P') \mid (n(x) . Q + Q') \rightarrow P \mid Q[m/x]$$

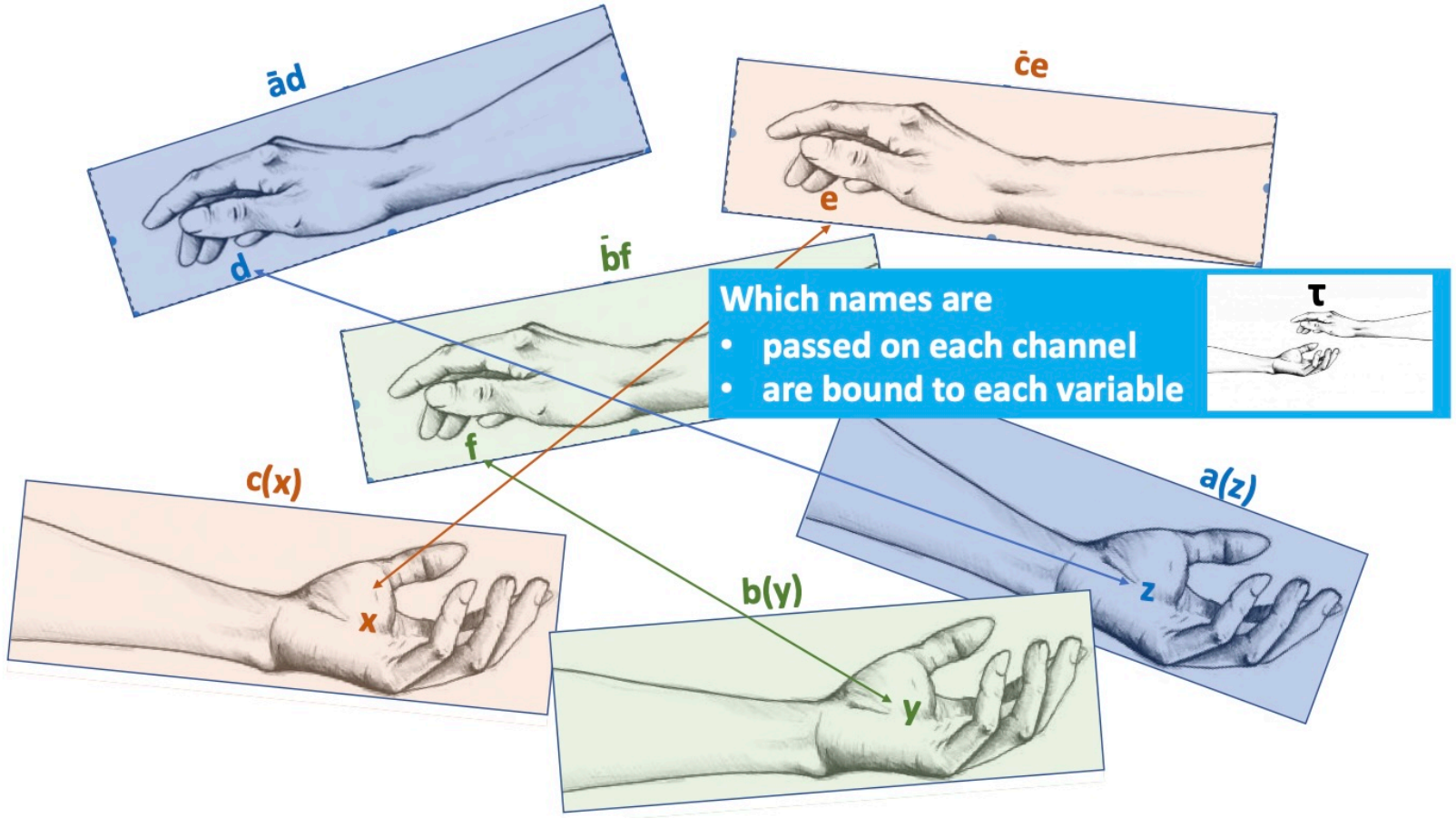
$$[Tau]\ (\tau . P + Q) \rightarrow P$$

$$[Res]\ \frac{P \rightarrow P'}{(\nu n)P \rightarrow (\nu n)P'} \qquad [Par]\ \frac{P \rightarrow P'}{P \mid Q \rightarrow P' \mid Q}$$

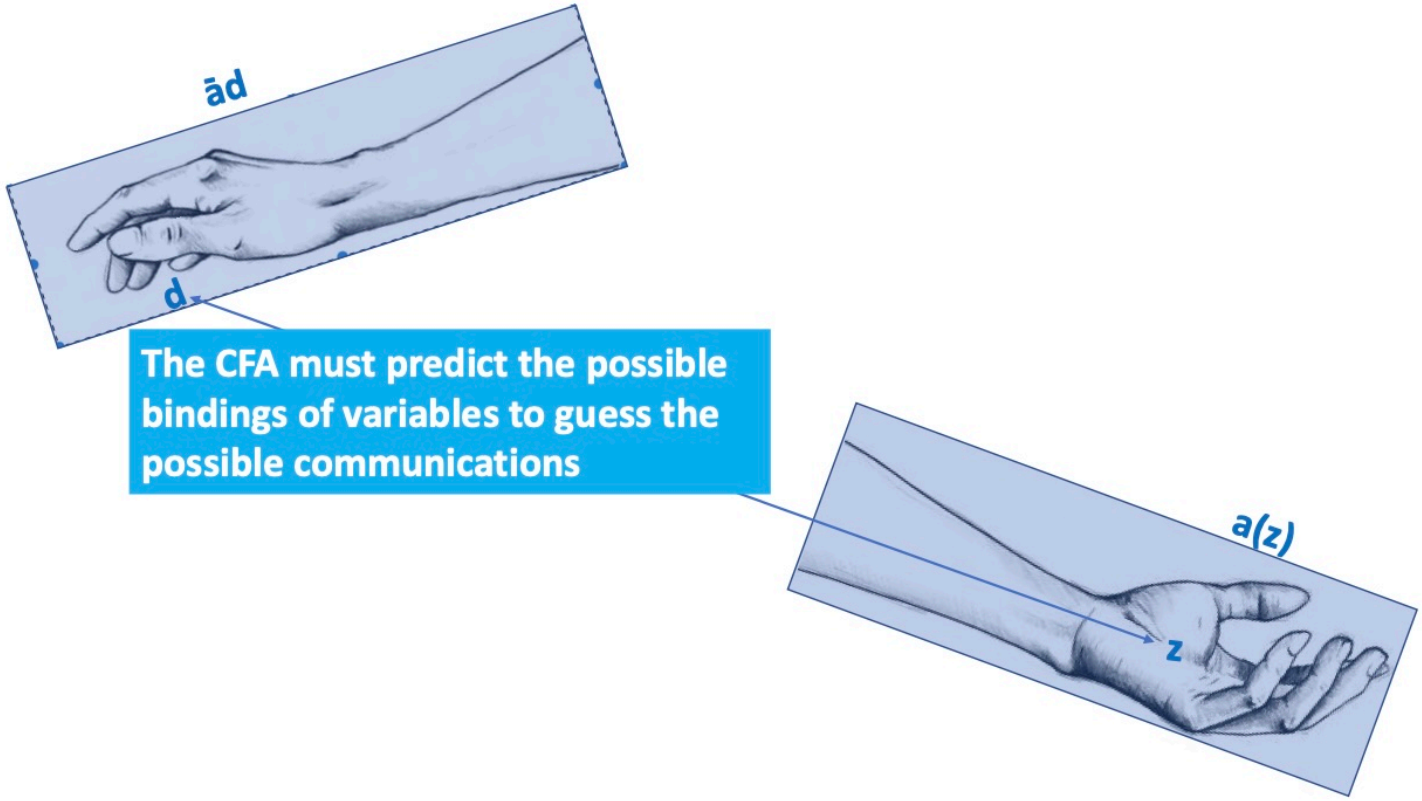
$$[Eq]\ \frac{P \equiv Q \quad Q \rightarrow Q' \quad Q' \equiv P'}{P \rightarrow P'}$$

# 0-CFA for π-calculus

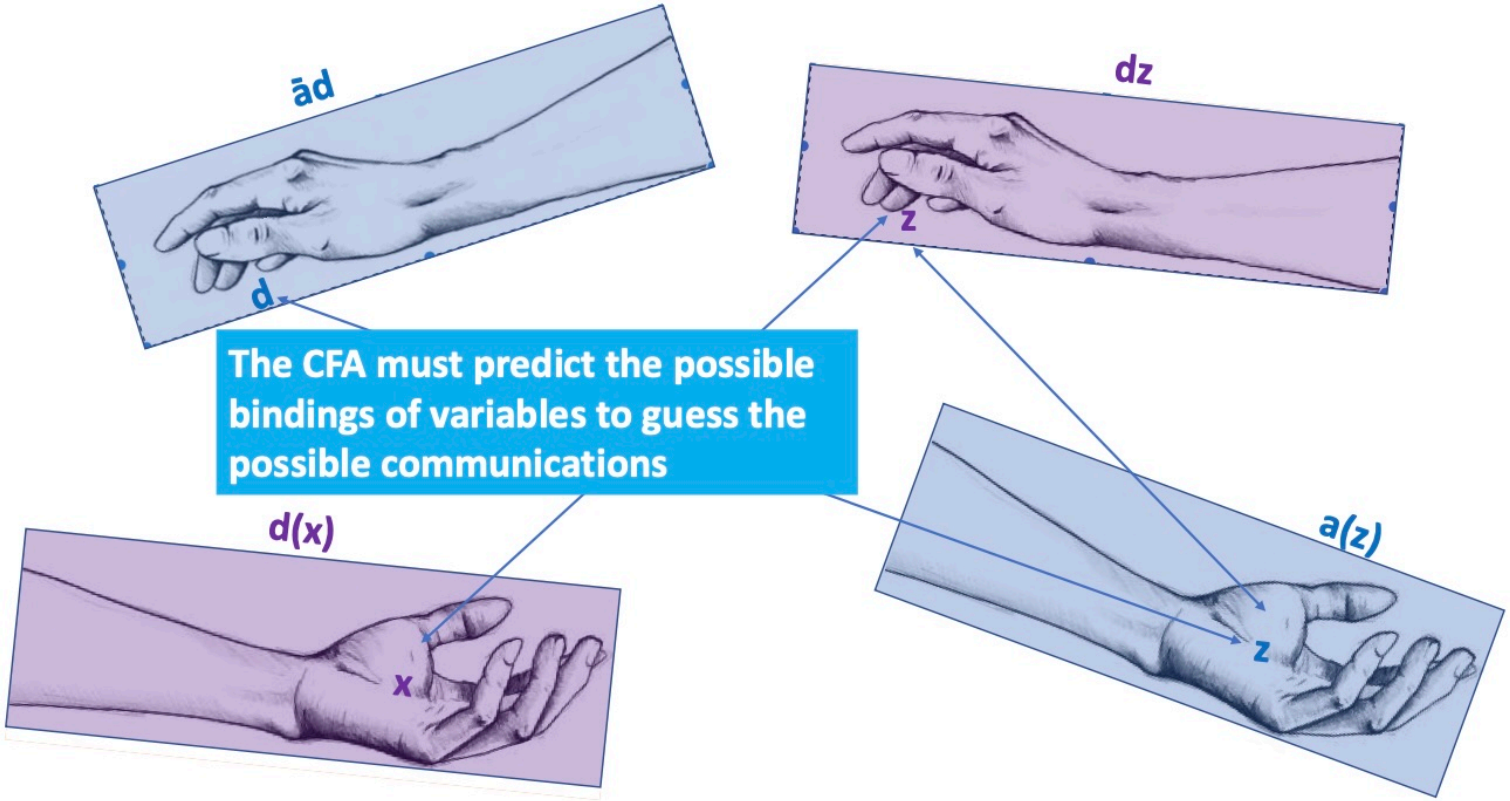
# CFA predicts communications



# CFA predicts communications



# CFA predicts communications



# Abstract domains: formally

How is abstract behaviour described?

As a pair of abstract domains given as functions $(\rho, \kappa)$

- **Abstract values** abstracts sets of constants
- An **abstract environment** $\rho$ associates a variable to the set of constants values it might be bound to
- An **abstract channel environment** $\kappa$ maps a (channel) constant to the set of constants that may be communicated over it
- $\rho$ and $\kappa$ and contain global information that captures information about the behaviour of the process $P_*$ under analysis

$$\mathbf{Val} = \wp(\mathbf{Const})$$
$$\rho : \mathbf{Var} \to \wp(\mathbf{Const})$$
$$\kappa : \mathbf{Var} \to \wp(\mathbf{Const})$$

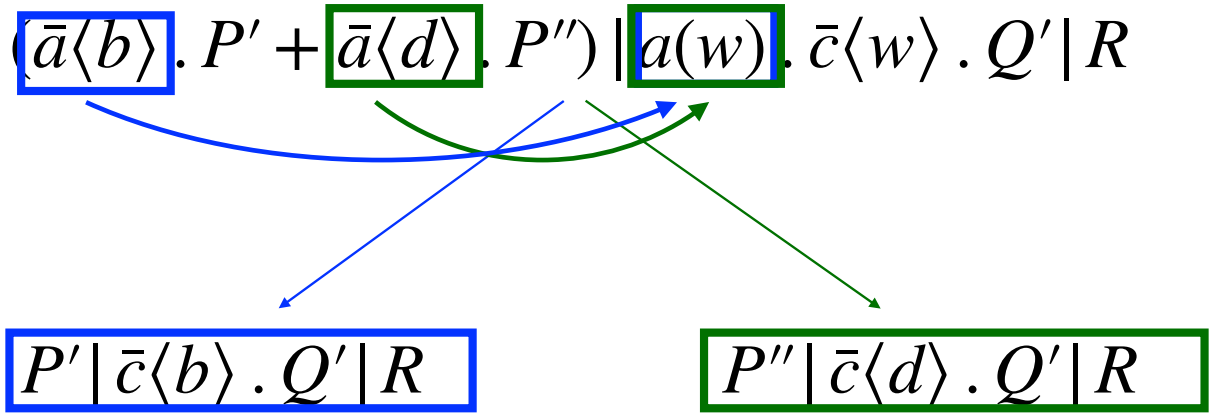
# Abstract domains (cont.)

- These domains can be turned into complete lattices by extending the subset ordering $\subseteq$ on $\wp(\mathbf{Const})$ in a pointwise manner

- We extend the **abstract environment** $\rho$ to **constants** by setting

$$\rho(n) = \{n\}$$

# Example 1




The analysis predicts:

- what can be exchanged on channels $a$ and $c$
- What can be bound to the variable $w$

In particular the analysis predicts **both** possible executions, e.g.,

$$\kappa(a) \supseteq \{b,d\}$$
$$\rho(w) \supseteq \{b,d\}$$

# The specification of a solution

Given a guess $(\rho, \kappa)$, we validate whether it is an acceptable CFA
A guess is acceptable if it safely over-approximates the runtime behavior

**Validation**
check clauses on logical judgements

- for processes

$$(\hat{\rho}, \hat{\kappa}) \vDash_P e$$

- for actions

$$(\hat{\rho}, \hat{\kappa}) \vDash_A \pi$$

Again we have exactly one clause for each of the syntactic forms of actions

# Clauses for actions: out

$$[out]\ (\rho, \kappa) \vDash_A \bar{u}\langle v \rangle \text{ iff } ?$$

Before introducing the general case with prefix $\bar{u}\langle v \rangle$, we introduce the simpler subcases, in which at least one between $u$ or $v$ is a constant:

- Case $\bar{u}\langle v \rangle \equiv \bar{n}\langle m \rangle$
- Case $\bar{u}\langle v \rangle \equiv \bar{n}\langle v \rangle$
- Case $\bar{u}\langle v \rangle \equiv \bar{u}\langle m \rangle$

# Clauses for actions: out

[Case $\bar{u}\langle v\rangle \equiv \bar{n}\langle m\rangle$]

- the rule must mimic that $m$ might be communicated over the channel
- How?
- By recording this in $\kappa$

$$[out]\ (\rho, \kappa) \vDash_A \bar{n}\langle m\rangle \text{ iff } m \subseteq \kappa(n)$$

In general $u$ as well as $v$ could contain variables so the righthand side of the clause has to be generalized

# Clauses for actions: out

[Case $\bar{u}\langle v\rangle \equiv \bar{n}\langle v\rangle$]

- the rule must mimic that whatever may be bound to $v$ might be communicated over the channel
- How?
- By looking inside $\rho(v)$, and
- By checking that all the contents in $\rho(v)$ are recorded in $\kappa$

$$[out]\ (\rho, \kappa) \vDash_A \bar{n}\langle v\rangle \text{ iff } \rho(v) \subseteq \kappa(n)$$

# Clauses for actions: out

[Case $\bar{u}\langle v \rangle \equiv \bar{u}\langle m \rangle$]

- the rule must mimics that $m$ may be communicated over any channel that can be bound to $u$
- How?
- By looking inside $\rho(u)$, and
- By checking that for all $n \in \rho(u)$ $m$ is recorded in $\kappa(n)$

$$[out]\ (\rho, \kappa) \vDash_A \bar{u}\langle m \rangle \text{ iff } \forall n \in \rho(u) : m \subseteq \kappa(n)$$

# Clauses for actions: out

$$[out]\ (\rho, \kappa) \vDash_A \bar{u}\langle v \rangle \text{ iff } \forall n \in \rho(u) : \rho(v) \subseteq \kappa(n)$$

# Clauses for actions: in

$[in]\ (\rho, \kappa) \vDash_A u(x)$ iff ?

Before introducing the general case with prefix $u(x)$, we introduce the simpler subcase where $u$ is a constant:

- Case $u(x) \equiv n(x)$

# Clauses for actions: in

Any possible constant may be input over $n$ here

[Case $u(x) \equiv n(x)$]

- the rule must mimics that $x$ might be bound to whatever may be communicated over the channel $n$ somewhere in the process
- How?
- By looking inside $\kappa(n)$, and
- Checking whether the contents in $\kappa(n)$ are included in $\rho(x)$

$$[in]\ (\rho, \kappa) \vDash_A n(x) \text{ iff } \kappa(n) \subseteq \rho(x)$$

$u$ could contain variables so the rhs of the clause has to be generalized

# Clauses for actions: in

$$[in]\ (\rho, \kappa) \vDash_A u(x) \text{ iff } \forall n \in \rho(u) : \kappa(n) \subseteq \rho(x)$$

# Clauses for actions: tau

$[tau]\ (\rho, \kappa) \vDash_A \tau$ iff true

# Clauses for processes: nil

$$[nil]\ (\rho, \kappa) \vDash_P \mathbf{0} \text{ iff true}$$

# Clauses for processes: par

Recursive calls

$$[par]\ (\rho, \kappa) \vDash_P P_1 \mid P_2 \text{ iff } (\rho, \kappa) \vDash_P P_1 \wedge (\rho, \kappa) \vDash_P P_2$$

# Clauses for processes: sum

Recursive calls

$$[sum]\ (\rho, \kappa) \vDash_P \Sigma_{i \in I} \pi_i\,.\,P_i \text{ iff } \forall i \in I : ((\rho, \kappa) \vDash_A \pi_1 \wedge (\rho, \kappa) \vDash_P P_i)$$

# Clauses for processes: match

The analysis is oblivious to the matching

$$[match]\ (\rho, \kappa) \vDash_P [x = y]P \text{ iff } (\rho, \kappa) \vDash_P P$$

# Clauses for processes: res

The analysis is oblivious to the scoping of constants

$$[res]\ (\rho, \kappa) \vDash_P (\nu n)P \text{ iff } (\rho, \kappa) \vDash_P P$$

# Clauses for processes: rep

The analysis is oblivious to the replication operator

$$[rep]\ (\rho, \kappa) \vDash_P\ !P \text{ iff } (\rho, \kappa) \vDash_P P$$

# CFA Clauses for π-calculus

$$[res]\ (\rho,\kappa) \models_P (\nu n)P \text{ iff } (\rho,\kappa) \models_P P$$

$$[par]\ (\rho,\kappa) \models_P P_1 \,|\, P_2 \text{ iff } (\rho,\kappa) \models_P P_1 \wedge (\rho,\kappa) \models_P P_2$$

$$[sum]\ (\rho,\kappa) \models_P \Sigma_{i\in I}\pi_i\,.\,P_i \text{ iff } \forall i \in I : ((\rho,\kappa) \models_A \pi_1 \wedge (\rho,\kappa) \models_P P_i)$$

$$[rep]\ (\rho,\kappa) \models_P\ !P \text{ iff } (\rho,\kappa) \models_P P$$

$$[match]\ (\rho,\kappa) \models_P [x = y]P \text{ iff } (\rho,\kappa) \models_P P$$

$$[nil]\ (\rho,\kappa) \models_P \mathbf{0} \text{ iff true}$$

---

$$[out]\ (\rho,\kappa) \models_A \bar{u}\langle v\rangle \text{ iff } \forall n \in \rho(u) : \rho(v) \subseteq \kappa(n)$$

$$[in]\ (\rho,\kappa) \models_A u(x) \text{ iff } \forall n \in \rho(u) : \kappa(n) \subseteq \rho(x)$$

$$[tau]\ (\rho,\kappa) \models_A \tau \text{ iff true}$$

# Back to Example 1

$$(\bar{a}\langle b\rangle . P' + \bar{a}\langle d\rangle . P'') \,|\, a(w) . \bar{c}\langle w\rangle . Q' \,|\, R$$

| | $\rho$ |
|---|---|
| $a$ | $\{a\}$ |
| $b$ | $\{b\}$ |
| $c$ | $\{c\}$ |
| $d$ | $\{d\}$ |
| $w$ | $\{b, d\}$ |

| | $\kappa$ |
|---|---|
| $a$ | $\{b, d\}$ |
| $b$ | $\emptyset$ |
| $c$ | $\{b, d\}$ |
| $d$ | $\emptyset$ |

# Back to Example 1

$(\rho,\kappa) \models (\bar{a}\langle b\rangle . P' + \bar{a}\langle d\rangle . P'') \mid a(w) . \bar{c}\langle w\rangle . Q' \mid R$ iff

$(\rho,\kappa) \models (\bar{a}\langle b\rangle . P' + \bar{a}\langle d\rangle . P'')$ and $(\rho,\kappa) \models a(w) . \bar{c}\langle w\rangle . Q'$ and $(\rho,\kappa) \models R$

$(\rho,\kappa) \models (\bar{a}\langle b\rangle . P' + \bar{a}\langle d\rangle . P'')$ iff $(\rho,\kappa) \models \bar{a}\langle b\rangle . P'$ and $(\rho,\kappa) \models \bar{a}\langle d\rangle . P''$

$(\rho,\kappa) \models \bar{a}\langle b\rangle . P'$ iff $(\rho,\kappa) \models \bar{a}\langle b\rangle \wedge (\rho,\kappa) \models P'$

$(\rho,\kappa) \models \bar{a}\langle b\rangle$ iff $\kappa(a) \subseteq \{b\}$

$(\rho,\kappa) \models \bar{a}\langle d\rangle . P''$ iff $(\rho,\kappa) \models \bar{a}\langle d\rangle$ and $(\rho,\kappa) \models P''$

$(\rho,\kappa) \models \bar{a}\langle d\rangle$ iff $\kappa(a) \subseteq \{d\}$

$(\rho,\kappa) \models a(w) . \bar{c}\langle w\rangle . Q'$ iff

$(\rho,\kappa) \models a(w)$ and $(\rho,\kappa) \models \bar{c}\langle w\rangle . Q'$

$(\rho,\kappa) \models a(w)$ iff $\kappa(a) \subseteq \rho(w)$

…

| | $\rho$ |
|---|---|
| $a$ | $\{a\}$ |
| $b$ | $\{b\}$ |
| $c$ | $\{c\}$ |
| $d$ | $\{d\}$ |
| $w$ | $\{b,d\}$ |

| | $\kappa$ |
|---|---|
| $a$ | $\{b,d\}$ |
| $b$ | $\emptyset$ |
| $c$ | $\{b,d\}$ |
| $d$ | $\emptyset$ |

# Exercise on the match clause

Any idea of taking care of matching?

$$[match]\ (\rho,\kappa) \vDash_P [x = y]P \text{ iff } (\rho(x) \cap \rho(y) \neq \varnothing \Rightarrow (\rho,\kappa) \vDash_P P)$$

The check proceeds only if the match can be successful, otherwise there is no point in analysing a sub-process that is not reachable

**Example**

- $a(x).c(y).[x = y].P' | \bar{a}\langle d\rangle.Q' | \bar{a}\langle b\rangle.Q'' | \bar{c}\langle b\rangle.R'$
  $\rho(x) = \{b, d\}$, $\rho(y) = \{b\}$ and $\rho(x) \cap \rho(y) = \{b\}$
- $a(x).c(y).[x = y].P' | \bar{a}\langle d\rangle.Q' | \bar{a}\langle b\rangle.Q'' | \bar{c}\langle f\rangle.R'$
  $\rho(x) = \{b, d\}$, $\rho(y) = \{f\}$ and $\rho(x) \cap \rho(y) = \varnothing$

# Theoretical properties

# Semantic correctness

The analysis information is a safe description of what will happen during the evaluation of the program, i.e., a sound over-approximation of the runtime behaviour

We can establish a subject reduction result stating that if we have an analysis result for $P$, and $P$ evolves into some process $Q$, then the very same analysis result is also valid for $Q$

# The nature of imprecision

This Control Flow Analysis is Context-Insensitive and is called 0-CFA

$$P_1 \triangleq a(x) \,|\, \bar{a}\langle b \rangle \qquad P_1 \triangleq a(x) + \bar{a}\langle b \rangle \qquad P_1 \triangleq a(x) \,.\, \bar{a}\langle b \rangle$$

- Note that the variable $y$ in $P_1$ can be bound to $b$, while in $P_2$ and in $P_3$ it cannot
- Instead, in the CFA estimate, we have that $\rho(x) \supseteq \{b\}$ in all the three cases

# Preserving the identity of names

Because of $\alpha$-renaming, we risk not to preserve the identity of names occurring in the process under analysis during the evaluation

We could use labels as done in the previous CFA or
introduce equivalence classes of names

- $\lfloor n \rfloor$ is the equivalence class corresponding to $n$; it is also called the *canonical name* of $n$
- $\lfloor x \rfloor = x$

We use a disciplined $\alpha$-renaming

- names can only be substituted with names in the same equivalence class

# Semantic correctness

We start from the process of interest $P_*$ and we lift $\lfloor\ \rfloor$ to obtain $\lfloor P_* \rfloor$

**Lemma**

If $P \equiv Q$ then $(\rho, \kappa) \vDash_P \lfloor P \rfloor$ iff $(\rho, \kappa) \vDash_P \lfloor Q \rfloor$

**Substitution lemma**

$(\rho, \kappa) \vDash_P \lfloor P \rfloor$ then $(\rho, \kappa) \vDash_P \lfloor P[m/x] \rfloor$, provided that $\lfloor m \rfloor \subseteq \rho(x)$

**Subject reduction result**

$(\rho, \kappa) \vDash_P \lfloor P \rfloor$ and $P \to Q$ then $(\rho, \kappa) \vDash_P \lfloor Q \rfloor$

# Existence

The set of proposed solutions can be partially ordered:
$(\rho, \kappa) \subseteq (\rho', \kappa')$ iff ordered pointwise

**Moore family**
The set $\{(\rho, \kappa) \vDash_P P\}$ is a Moore family for all processes $P$

Therefore there always exists a least (best) solution $(\rho, \kappa)$
obtained as $\sqcap \{(\rho, \kappa) \vDash_P P\}$

# Implementation

There is a constructive procedure for obtaining the least solution of low polynomial complexity

Also in this CFA the acceptability judgement associates each process with a set of logical constraints that must be solved

One obvious approach is to develop a special purpose constraint solver handling exactly the combination of set inclusions and first order logic that we have used; essentially this is the approach taken for the previous CFA

An alternative, and more general approach, is to transform the constraints into an appropriate fragment of first order logic and then use an off-theshelf solver

# Variation

# Polyadic π-calculus: syntax

$\pi ::=$ $\overline{u}\langle\vec{v}\rangle$ // send $v$ on $u$

$\mid$ $u(\vec{x})$ // receive on $u$ and store in $v$

$\mid$ $\tau$ // internal action

Polyadic send and receive

$P ::=$ $\mathbf{0}$ // inactive process

$\mid$ $\pi \,.\, P$ // action prefix

$\mid$ $[x = y]P$ // matching (like, if $x = y$ then $P$)

$\mid$ $(\nu n)P$ // restriction (like $n$ only known in $P$)

$\mid$ $\Sigma_{i \in I} \pi_i \,.\, P_i$ // non deterministic choice

$\mid$ $P_1 \mid P_2$ // parallel composition

$\mid$ $!P$ // replication (like $P \mid P \mid P \mid P \mid \ldots$)

# Reduction semantics

$$[Com]\ (\overline{n}\langle\overrightarrow{m}\rangle\,.\,P + P') \mid (n(\overrightarrow{x})\,.\,Q + Q') \to P \mid Q[\overrightarrow{m}/\overrightarrow{x}] \text{ if } |\overrightarrow{m}| = |\overrightarrow{x}|$$

now arity matters

$$[Tau]\ (\tau\,.\,P + Q) \to P$$

$$[Res]\ \frac{P \to P'}{(\nu n)P \to (\nu n)P'} \qquad [Par]\ \frac{P \to P'}{P \mid Q \to P' \mid Q}$$

$$[Eq]\ \frac{P \equiv Q \quad Q \to Q' \quad Q' \equiv P'}{P \to P'}$$

# Abstract domains

- An **abstract environment** $\rho$ associates a variable to the set of constants values it might be bound to
- An **abstract channel environment** $\kappa$ maps a (channel) constant to the set of sequences of constants that may be communicated over it
- An **error component** $\psi$ records the set of (channel) constants where there may be an arity mismatch in a communication
- $\rho$ and $\kappa$ and contain global information that captures information about the behaviour of the process $P_*$ under analysis, while $\psi$ captures local information

$$\mathbf{Val} = \wp(\mathbf{Const})$$
$$\rho : \mathbf{Var} \to \wp(\mathbf{Const})$$
$$\kappa : \mathbf{Var} \to \wp(\mathbf{Const})$$
$$\psi : \wp(\mathbf{Const})$$

# The specification of a solution

Given a guess $(\rho, \kappa, \psi)$, we validate whether it is an acceptable CFA
A guess is acceptable if it safely over-approximates the runtime behavior

**Validation**
check clauses on logical judgements

- for processes

$$(\hat{\rho}, \hat{\kappa}) \vDash_P e : \psi$$

- for actions

$$(\hat{\rho}, \hat{\kappa}) \vDash_A \pi : \psi$$

Again we have exactly one clause for each of the syntactic forms of actions

# Clauses for actions: out

$$[out]\ (\rho, \kappa) \vDash_A \bar{u}\langle\vec{v}\rangle : \psi \text{ iff } \forall n \in \rho(u) : \rho(\vec{v}) \subseteq \kappa(n)$$

# Clauses for actions: in

$$[in]\ (\rho,\kappa) \vDash_A u(x) : \psi \text{ iff } \forall n \in \rho(u) : \kappa(n) \cap \mathbf{Const}^{|\vec{x}|} \subseteq \rho(\vec{x}) \wedge \kappa(n) \setminus \mathbf{Const}^{|\vec{x}|} \neq \emptyset \Rightarrow n \in \psi$$

# Clauses for processes: par

$$[par]\ (\rho,\kappa) \vDash_P P_1 \,|\, P_2 : \psi \text{ iff } (\rho,\kappa) \vDash_P P_1 : \psi_1 \wedge (\rho,\kappa) \vDash_P P_2 : \psi_2 \wedge \psi_1 \cup \psi_2 \subseteq \psi$$

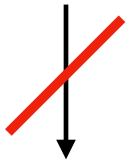
# Clauses for processes: sum

$$[sum]\ (\rho,\kappa) \vDash_P \Sigma_{i\in I}\pi_i\,.\,P_i : \psi \text{ iff } \forall i \in I : ((\rho,\kappa) \vDash_A \pi_1 : \psi_1' \wedge \psi_1' \subseteq \psi \wedge (\rho,\kappa) \vDash_P P_i : \psi_1 \wedge \psi_1 \subseteq \psi)$$

# Example 2

$$\bar{c}\langle n\rangle \,.\, \bar{c}\langle n\rangle \mid c(x) \,.\, \bar{x}\langle m\rangle \mid c(x,y) \,.\, x(y)$$

$$\bar{c}\langle n\rangle \mid \bar{n}\langle m\rangle \mid c(x,y) \,.\, x(y)$$



The process stucks when trying to communicate over $c$

# Example 2

$$\bar{c}\langle n\rangle \,.\, \bar{c}\langle n\rangle \mid c(x) \,.\, \bar{x}\langle m\rangle \mid c(x,y) \,.\, x(y)$$

| | $\rho$ |
|---|---|
| $x$ | $\{n\}$ |
| $y$ | $\{m\}$ |

| | $\kappa$ |
|---|---|
| $c$ | $\{n\}$ |
| $n$ | $\{m\}$ |
| $m$ | $\emptyset$ |

$$\psi = \{c\} \neq \emptyset$$

The analysis reflects that the process will indeed become stuck when trying to communicate over $c$

# Applications

# Applications of the CFA

We shall now give a few applications of the preceding analysis and present the associated adequacy results

Our approach is:

- first we define a notion of the dynamic property based on the semantics,
- then we define a notion of the corresponding static property based on the analysis and
- finally we prove an *adequacy result* expressing that the static property implies the dynamic one

# Contexts

A context $C$ is formally given by

$$C ::= (\nu n)C \mid C|P \mid [\,.\,]$$

A context essentially is a process with a hole [ . ] where another process can be inserted

- we write $C[P]$ for the process obtained by inserting $P$ in the hole of $C$

# Well-behavedness

Well-behavedness guarantees that communicating actions have matching arities

- The process $P_*$ is dynamically well-behaved if whenever $P$ evolves into a process that is structurally congruent to one of the form $C[(\overline{n}\langle\overrightarrow{m}\rangle \,.\, R + R') \mid (n(\vec{x}) \,.\, Q + Q')]$ for some context $C$ then $|\overrightarrow{m}| = |\vec{x}|$

context

- The process $P_*$ is statically well-behaved if there exist $\rho$ and $\kappa$ such that

$$(\rho, \kappa) \vDash_P P_* : \varnothing$$

meaning there is a valid analysis result for $P_*$ where the error component $\psi$ is empty

# Well-behavedness (cont.)

**Theorem (Adequacy for well-behaved processes)**

If the process $P_*$ is statically well-behaved then it is also dynamically well-behaved

the analysis guarantees that no runtime communication mismatch can occur

# Well-sortedness

**Idea**: Channels are not all interchangeable: each channel carries values of a specific shape

In $\pi$-calculus, any name can be: sent, received, used as a channel, or communicated over any other channel

Extremely expressive, but also too permissive, e.g., we could allow both

$$a\langle b\rangle \text{ and } a\langle b, c\rangle$$

on the same channel $a$

To discipline communication we can use sorts

A sort specifies

- the arity of the communicated tuples, and
- the sorts of their components (i.e., what kind of tuples can be communicated on a channel)

# Well-sortedness: example

If the sort of $a$ is $(S_1, S_2)$ then

- $a$ is a binary channel,
- the first communicated name must have sort $S_1$ and
- the second communicated name must have sort $S_2$

For instance

$$a\langle b, c\rangle$$

is well-sorted iff

$$\sigma(b) = S_1 \text{ and } \sigma(c) = S_2$$

# Well-sortedness (cont.)

We have a set of sorts and require that each constant $n$ has a sort

$$\Sigma(n) \in Sort$$

- A sorting is defined as a function

$$\Sigma : Sort \to Sort*$$

that for each sort specifies a sequence of sorts describing not only the arity of the constants with that sort but also the sorts of the constants that may be communicated over it

- The sorts are preserved by disciplined α-renaming so if $\lfloor n \rfloor = \lfloor m \rfloor$ also $\sigma(n) = \sigma(m)$
- We extend $\sigma$ in a component-wise manner to work on tuples as well as sets of tuples (sorting can the be checked componentwise)

# Well-sortedness

- The process $P_*$ is dynamically well-sorted if whenever $P$ evolves into a process that is structurally congruent to one of the form $C[(\overline{n}\langle\overrightarrow{m}\rangle . R + R') \mid (n(\vec{x}) . Q + Q')]$ for some context $C$ then $|\overrightarrow{m}| = |\vec{x}|$ as well as $\Sigma(\sigma(m)) = \sigma(\overrightarrow{m})$

- The process $P_*$ is statically well-sorted if there exist $\rho$ and $\kappa$ such that

$$(\rho, \kappa) \vDash_P P_* : \varnothing$$

and furthermore, for all constants $n$,

$$\sigma(\kappa(n)) \subseteq \{\Sigma(\sigma(\vec{n}))\}$$

abstract values are tuples of names

everything CFA predicts for channel n has the correct shape prescribed by Σ

# Well-sortedness (cont.)

**Theorem (Adequacy for well-sorted processes)**

If the process $P_*$ is statically well-sorted then it is also dynamically well-sorted

again, the analysis guarantees that no runtime arity mismatch can occur

# Security properties

# Secrecy-preserving processes

We classify names as secret and public and use the analysis to impose a confidentiality policy on the processes, ensuring that they do not send secret constants on public channels

We introduce a mapping $\Sigma : \mathbf{Const} \to \{\text{secret}, \text{public}\}$
We assume that if $\lfloor n \rfloor = \lfloor m \rfloor$ then $\Sigma(n) = \Sigma(m)$
We extend $\Sigma$ in a component-wise manner to work on tuples

**Property**: a secret channel cannot be sent on a public channel


BODEI, C., DEGANO, P., NIELSON, F., AND NIELSON, H. R.

- 1998. *Control flow analysis for the π-calculus*. In Proceedings of the International Conference on Concurrency Theory (CONCUR). Lecture Notes in Computer Science, vol. 1466, Springer, 84–98.
- 1999. *Static analysis of processes for no read-up and no write-down*. In Proceedings of the International Conference on Foundations of Software Science and Computation Structures (FOSSACS). Lecture Notes in Computer Science, vol. 1578, Springer, 120–134.
- 2001a. *Static analysis for the π-calculus with applications to security*. Inform. Comput. 168, 1, 68–92.

# Secrecy-preserving processes

**Dynamic property**

The process $P_*$ is dynamically secrecy-preserving if whenever $P_*$ evolves into a process that is structurally congruent to one of the form $C[(\overline{n}\langle\overrightarrow{m}\rangle . P + P') \mid (n(\vec{x}) . Q + Q')]$, then $|\overrightarrow{m}| = |\vec{x}|$ as well as

$$\Sigma(n) = \text{public} \Rightarrow \Sigma(\overrightarrow{m}) = \text{public}$$

**Static property**

The process $P_*$ is statically secrecy-preserving if

- there exists $\rho$ and $\kappa$ such that $(\rho, \kappa) \vDash_P P_*$ and
- $\kappa$ is such that

$$\Sigma(n) = \text{public} \Rightarrow \forall \overrightarrow{m} \in \kappa(n) : \Sigma(\overrightarrow{m}) = \text{public}$$

# Secrecy-preserving processes

**Theorem (Adequacy for secrecy-preserving processes)**
If the process $P_*$ is statically secrecy-preserving
then it is also dynamically secrecy-preserving

The analysis guarantees that no secret information can flow through public channels

# Non-leaking processes

We assign security levels to the channels and use the analysis to impose a confidentiality policy on the processes, ensuring that they do not send constants with a high security level on channels with a lower security level

For simplicity we only impose two levels, low and high (low $\sqsubseteq$ high), and we want to ensure that only low information is communicated on low channels

We introduce a mapping $\Lambda : \mathbf{Const} \to \{\mathsf{low}, \mathsf{high}\}$ that to each constant associates a *security level*

security lattice

We extend $\Lambda$ in a component-wise manner to work on tuples

We assume that if $\lfloor n \rfloor = \lfloor m \rfloor$ then $\Lambda(n) = \Lambda(m)$

**Property**: the level of the msg cannot be higher than the level of the channel


BODEI, C., DEGANO, P., NIELSON, F., AND NIELSON, H. R.

- 1998. *Control flow analysis for the π-calculus*. In Proceedings of the International Conference on Concurrency Theory (CONCUR). Lecture Notes in Computer Science, vol. 1466, Springer, 84–98.
- 1999. *Static analysis of processes for no read-up and no write-down*. In Proceedings of the International Conference on Foundations of Software Science and Computation Structures (FOSSACS). Lecture Notes in Computer Science, vol. 1578, Springer, 120–134.
- 2001a. *Static analysis for the π-calculus with applications to security*. Inform. Comput. 168, 1, 68–92.

# Non-leaking processes

**Dynamic property**

The process $P_*$ is dynamically non-leaking if

whenever $P_*$ evolves into a process that is structurally congruent to one of the form $C[(\overline{n}\langle\overrightarrow{m}\rangle . P + P') \mid (n(\vec{x}) . Q + Q')]$ then $|\overrightarrow{m}| = |\vec{x}|$ as well as $\Lambda(\overrightarrow{m}) \sqsubseteq \Lambda(n)^{|\overrightarrow{m}|}$

**Static property**

The process $P_*$ is statically non-leaking if

- there exists $\rho$ and $\kappa$ such that $(\rho, \kappa) \models_P P_*$ and
- $\kappa$ is such that $\forall \overrightarrow{m} \in \kappa(n) : \Lambda(\overrightarrow{m}) \sqsubseteq \Lambda(n)^{|\overrightarrow{m}|}$ for all constants $n$

Every value predicted by the CFA for channel $n$ must be allowed by the security policy

So the CFA cache becomes an approximation of possible information flows

the abstract cache κ becomes a lightweight information-flow analysis

# Non-leaking processes

**Theorem (Adequacy for non-leaking processes)**
If the process $P_*$ is statically non-leaking
then it is also dynamically non-leaking

The analysis guarantees absence of runtime leaks

# Taint analysis

We classify names as tainted and untainted
The analysis propagates values together their taint state and operations may transform taint information

The previous confidentiality properties were channel-oriented

i.e., focussed on what may flow through a channel

Taint analysis is instead data-oriented, i.e., focussed on how information propagates through computations

Operations may propagate or transform taint information:

- combining values (e.g., to form a tuple of values) may propagate taint (simple aggregation of values can be “infected” by the presence of tainted values)
- “sanitisation” functions, special functions able to untainted the values

# Taint analysis

We introduce a mapping $\Theta : \mathbf{Const} \to \{\text{tainted}, \text{untainted}\}$
We assume that if $\lfloor n \rfloor = \lfloor m \rfloor$ then $\Theta(n) = \Theta(m)$
We extend $\Theta$ in a component-wise manner to work on tuples

A **sink** is a program point or channel where tainted data must not arrive, e.g., a channel where confidential or untrusted data must not flow

**Property**: a tainted value should never reach a sensitive sink

# Taint safety

**Dynamic property**

The process $P_*$ is dynamically taint-safe if

whenever $P_*$ evolves into a process that is structurally congruent to one of the form $C[(\overline{n}\langle\overrightarrow{m}\rangle . P + P') \mid (n(\vec{x}) . Q + Q')]$ and $n$ is a sensitive sink channel then $|\overrightarrow{m}| = |\vec{x}|$ as well as

$$\Theta(\overrightarrow{m}) = \{\text{untainted}\}$$

**Static property**

The process $P_*$ is statically taint-safe if

- there exists $\rho$ and $\kappa$ such that $(\rho, \kappa) \vDash_P P_*$ and for every sensitive sink channel $n$
- $\kappa$ is such that

$$\forall \overrightarrow{m} \in \kappa(n) : \Theta(\overrightarrow{m}) = \{\text{untainted}\}$$

for all constants $n$

# Taint-safe processes

**Theorem (Adequacy for taint-safe processes)**
If the process $P_*$ is statically taint-safe
then it is also dynamically taint-safe

The analysis guarantees that tainted information cannot reach sensitive sinks

# Taint safety: example

A tainted value should never reach a sensitive sink

$P \equiv input(x)\,.\,clean\langle sanitize(x)\rangle \mid clean(y)\,.\,sql\langle y\rangle$

with:

- $input$ = untrusted source,
- $sql$ = sensitive sink,
- $sanitize$ = sanitisation function

Assume

$\Theta(input) = \text{tainted}$

$\Theta(sanitize(v)) = \text{untainted}$ for every value $v$

In this example, no tainted value reaches the sink $sql$, due to $sanitize$

# Taint safety: example

A tainted value should never reach a sensitive sink

$P \equiv input(x)\,.\,clean\langle sanitize(x)\rangle \mid clean(y)\,.\,sql\langle y\rangle$

$\Theta(input) = \text{tainted}$
$\Theta(sanitize(v)) = \text{untainted}$ for every value $v$

The CFA predicts:

$\kappa(clean) = \{\text{untainted}\}$

$\kappa(sql) = \{\text{untainted}\}$

So the static property $\forall \overrightarrow{m} \in \kappa(n) : \Theta(\overrightarrow{m}) = \{\text{untainted}\}$ holds

# Taint safety: example

Without sanitisation:

$P' \equiv input(x) \,.\, sql\langle x \rangle$

The CFA predicts:

$\kappa(sql) = \{\text{tainted}\}$

So the static property $\forall \overrightarrow{m} \in \kappa(n) : \Theta(\overrightarrow{m}) = \{\text{untainted}\}$ does **not** holds and the process is not statically taint-safe.

# Taint analysis

Unlike secrecy: taint is usually not a lattice over channels, but over data dependencies

Taint follows values, not channels

# CFA pattern

We have seen many examples of CFA: we can now identify a common pattern

- Choose those values of interest for the language and define the shape of solutions
- Prove that all estimates are semantically correct
- Prove that least estimate exists
- Derive a constructive procedure that builds estimates
- Select a specific dynamic property and define a static check on solutions
  This may require to refine estimates

# CFA vs Type Systems

- Type Systems are prescriptive, i.e. they infer types and impose the well-formedness conditions at the same time
- Control Flow Analysis is mainly descriptive, i.e. it merely infers the information and then leaves it to a separate step to actually impose demands on when programs are well-formed
- For each property, it is often the case that: a new ad hoc type system is necessary, while only a new test on the same CFA analysis is needed

# Control Flow Analysis for the π-calculus

Control Flow Analysis provides a static approximation of:

- possible communications,
- exchanged tuples,
- dynamic process interactions.

Starting from communication flows, we refined the analysis to enforce different properties

- The same CFA framework can therefore be enriched to verify different classes of communication and security properties
- Static approximations of communication flows become static guarantees about runtime behaviour

# Conclusion: chasing the flow

Control Flow Analysis computes a sound approximation of:

- how values flow,
- where functions may be applied,
- how processes may interact

All the analyses studied rely on:

- abstraction
- over-approximation
- fixpoint computation
- semantic soundness

Control Flow Analysis is not only about predicting control flow:
it provides a general framework for reasoning statically about dynamic behaviour


The End